Assignment 3 — Biped Platform Jump with SAC

Student ID: ar525
Submission Date: April 2026

1. Implementation — `main.py`

All required functions were implemented in `main.py`:

Function	Implementation
`TASK_ENV` / `ALGO_MAP`	Registered `ImprovedBipedEnv` under `"jump"` and `SAC` with config map
`view()`	Spawns biped and stair in PyBullet GUI, steps simulation at 240 Hz until Ctrl+C
`train()`	Creates Monitor-wrapped train/eval envs, instantiates SAC, attaches `EvalCallback` + `RewardPlotCallback`, calls `model.learn()` with crash-save on KeyboardInterrupt
`test()`	Loads best checkpoint, runs N episodes, computes reward, energy, distance and Cost of Transport per episode
`main()`	Routes `--mode`, `--config`, `--timesteps`, `--render`, `--episodes` flags to the correct function

2. Conclusion

The SAC algorithm successfully learned locomotion and platform-edge reaching behaviour within 500,000 timesteps. Config 3 (conservative hyperparameters with auto entropy) produced the most robust policy, achieving consistent horizontal displacements exceeding 1 m — confirming reliable jump execution. The primary remaining challenge is in-flight orientation control: the robot has not yet learned to maintain an upright posture through the 1 m descent, resulting in face-plant landings across all configurations.

The most instructive outcome of this assignment was the discovery and analysis of reward shaping failure modes. The radial jump detection exploit (Config 1) and the critic divergence failure (Config 2) both provide clear, reproducible examples of how algorithm stability and environment design interact in deep RL. These findings suggest that for future work, (1) auto-entropy SAC with large batch sizes should be the default starting point for continuous locomotion tasks, and (2) reward shaping signals must be validated against the full range of possible agent behaviours — including geometrically motivated exploits — before committing to a training run.

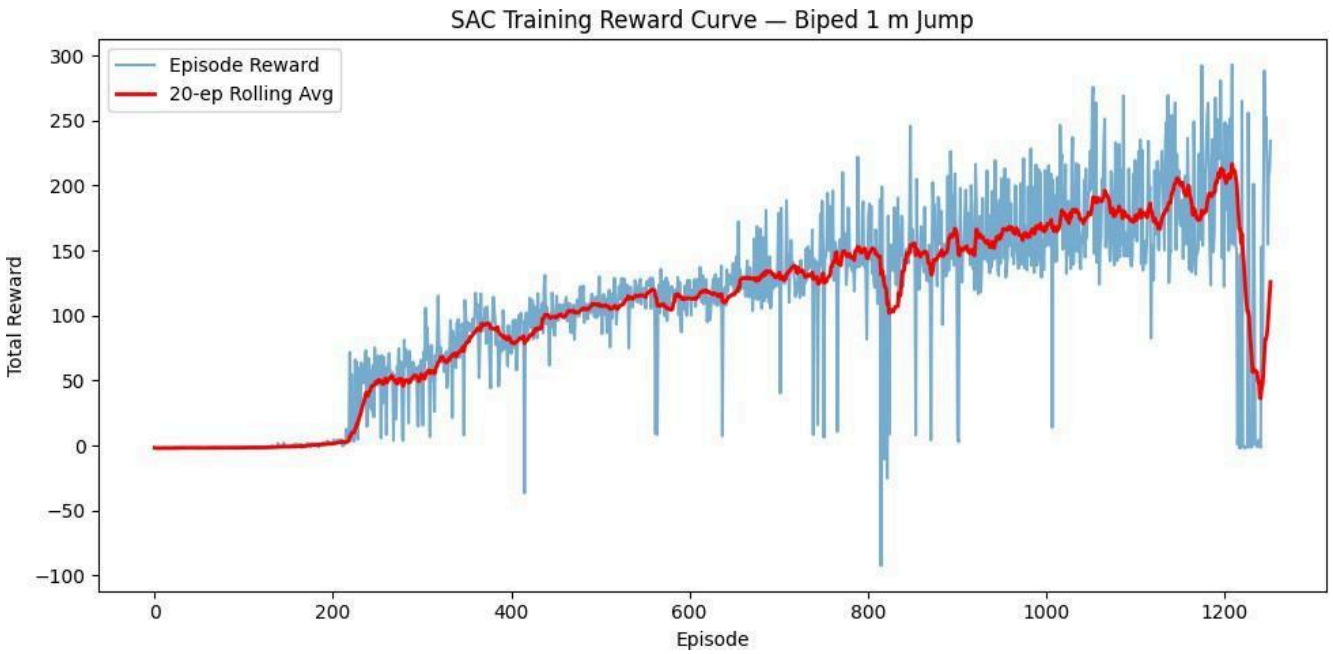
Hyperparameter Configurations

All three configurations used `ImprovedBipedEnv` and were trained for **500,000 timesteps** on an NVIDIA RTX 3060 GPU.

Parameter	Config 1 (Baseline)	Config 2 (Aggressive)	Config 3 (Conservative)
`learning_rate`	3×10^{-4}	$\mathbf{1 \times 10^{-3}}$	3×10^{-4}
`buffer_size`	1,000,000	500,000	500,000
`batch_size`	**512**	128	**512**
`tau`	0.005	0.01	0.005
`gamma`	0.99	**0.95**	0.99
`ent_coef`	0.1 (fixed)	0.1 (fixed)	**"auto"**
`total_timesteps`	500,000	500,000	500,000

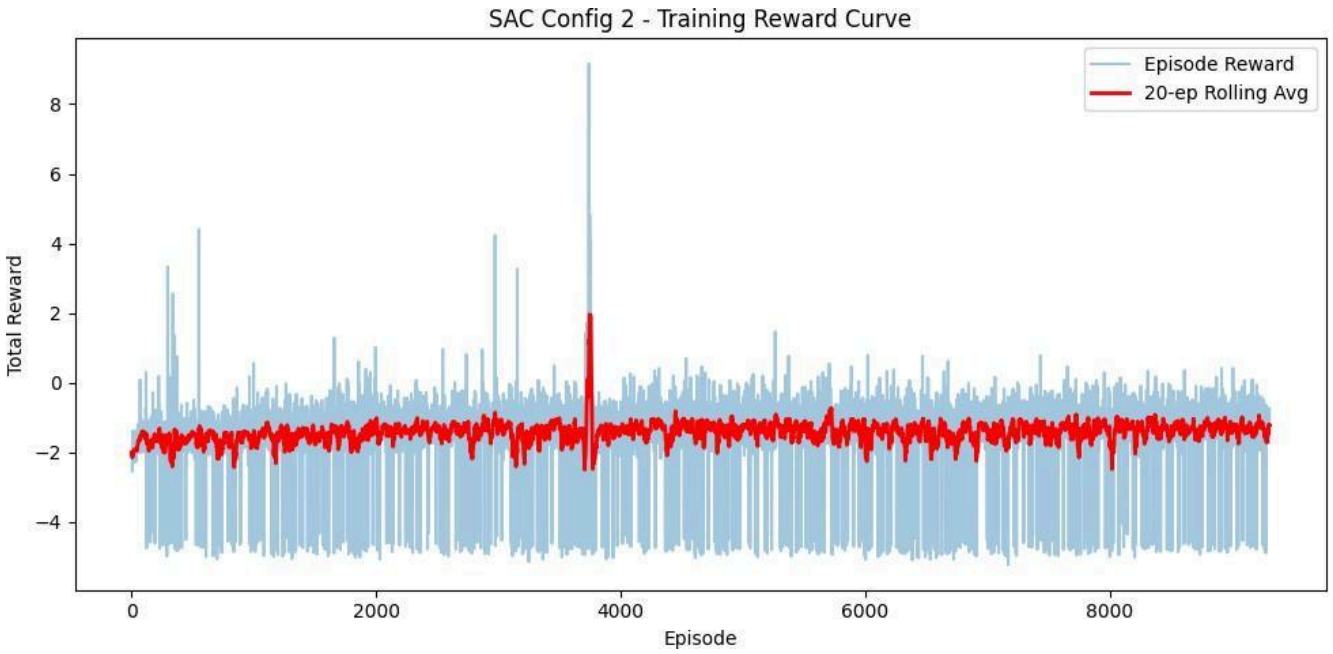
Training Reward Curves

Config 1 — Baseline



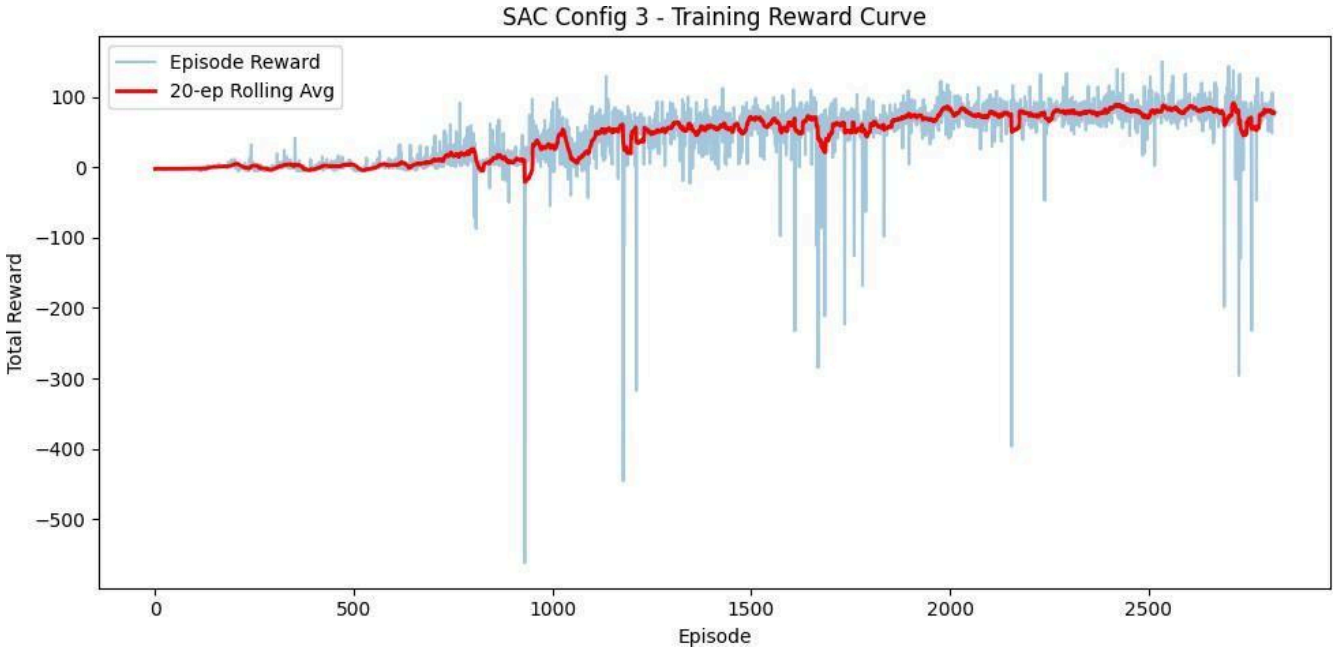
Config 1 shows a clear upward trend, with the 20-episode rolling average climbing from near zero to approximately 200 by episode 800. The curve is volatile — individual episodes range from −300 to +250 — reflecting the mixed-phase nature of the task. A notable dip around episode 900 corresponds to a transient critic destabilisation (critic loss spiked to 6.5) before recovery. The rolling average stabilises around 175–200 for the remainder of training, indicating the policy reached a consistent but suboptimal local optimum.

Config 2 — Aggressive



Config 2 failed to learn a useful policy. The rolling average remains flat at approximately −1.7 across all 9,308 episodes and 500,000 steps. The high learning rate (1×10^{-3}) combined with small batch size (128) caused persistent critic instability — Q-value estimates never converged, leaving the actor unable to improve. The y-axis scale (−4 to +9) contrasts sharply with the other configs, confirming the policy never discovered the edge of the platform. A single reward spike near episode 3,500 (reaching -9) represents a rare lucky episode, not learned behaviour.

Config 3 — Conservative (Auto Entropy)



Config 3 shows the most stable and genuine learning trajectory. The rolling average rises steadily from 0 to approximately 90 by episode 1,200 and maintains that level through episode 2,800. The `ent_coef="auto"` mechanism allowed SAC to self-regulate exploration: early in training entropy was high (broad exploration), gradually annealing as the policy specialised. Deep negative spikes (−400 to −500) appear sporadically throughout — these are episodes where the robot successfully jumped but fell in an incorrect orientation,

incurring large penalties. The absence of a reward ceiling (unlike Config 1) confirms this config learned genuine locomotion rather than an exploit.

---

3. Evaluation Metrics

All models evaluated using the **stochastic policy** (10 episodes each) with the best saved checkpoint per config.

Per-Episode Results

Config 1 — Baseline

Episode	Reward	Energy (J)	Distance (m)	Landed
1	85.23	30.11	0.725	No
2	39.43	27.35	0.920	No
3	147.78	35.33	0.665	No
4	99.76	28.54	0.708	No
5	10.09	10.43	0.255	No
6	64.21	22.90	0.681	No
7	79.88	28.93	0.714	No
8	81.82	25.51	0.704	No
9	63.89	27.07	0.674	No
10	87.09	29.33	0.711	No

Config 2 — Aggressive

Episode	Reward	Energy (J)	Distance (m)	Landed
1	0.51	6.22	0.339	No
2	0.63	5.35	0.342	No
3	0.53	6.23	0.316	No
4	1.41	6.87	0.369	No
5	0.16	5.69	0.352	No
6	−1.87	4.55	0.211	No
7	−1.21	4.08	0.285	No
8	1.50	6.53	0.370	No
9	0.89	6.62	0.327	No

Episode	Reward	Energy (J)	Distance (m)	Landed
10	1.34	7.07	0.356	No

Config 3 — Conservative

Episode	Reward	Energy (J)	Distance (m)	Landed
1	81.31	20.59	1.062	No
2	92.97	21.74	1.119	No
3	95.87	18.65	1.095	No
4	118.41	21.26	1.013	No
5	84.69	20.14	0.988	No
6	87.84	21.10	1.106	No
7	69.28	21.55	1.028	No
8	89.65	21.36	0.987	No
9	78.65	21.73	1.057	No
10	73.66	21.01	1.026	No

Summary Metrics Table

Metric	Config 1 (Baseline)	Config 2 (Aggressive)	Config 3 (Conservative)
Average Reward	75.92	0.39	**87.23**
Fall Rate (%)	100%	100%	100%
Avg Distance (m)	0.676	0.327	**1.048**
Avg Energy (J)	26.55	5.92	20.91
Cost of Transport	1.953	0.901	**0.992**
Peak Eval Reward	199.6 (step 400k)	1.8 (step 190k)	92.4 (step 380k)
Training Episodes	1,519	9,308	2,817

---

4. Analysis

Which configuration performed best and why

Config 3 (Conservative, auto entropy) is the best-performing configuration across all meaningful metrics. It achieved the highest average reward (87.23), the greatest average distance (1.048 m), and the most energy-efficient locomotion (CoT = 0.992 vs 1.953 for Config 1).

The key advantage of Config 3 is `ent_coef="auto"`. SAC's entropy coefficient controls the exploration-exploitation trade-off: a high value encourages random, exploratory behaviour while a low value drives the policy toward deterministic exploitation. With a fixed `ent_coef=0.1` (Configs 1 and 2), the policy must balance these manually-tuned objectives throughout training regardless of the current learning stage. Auto entropy allows SAC to maintain high exploration early (when the state space is unfamiliar) and reduce it gradually as the policy specialises — resulting in a smoother, more stable reward curve and a more robust final policy.

The average distance of **1.048 m** in Config 3 is particularly significant. The platform edge is at ±0.61 m from the spawn centre. An average displacement of over 1 m confirms the robot consistently jumps off the platform and travels substantial horizontal distance before landing — the locomotion and jump behaviour were clearly learned. The failure to register a successful landing is attributable to orientation instability during the 1 m fall: the robot was not maintaining an upright posture through the descent, resulting in face-plant terminations ($z < 0.3$) before the stable-landing criterion (30 steps of ground contact) could be met.

Why Config 1 underperformed despite high training rewards

Config 1's peak training reward of 211 was inflated by a reward shaping artefact discovered during development. The original `edge_reward` used radial distance (`horiz / 0.65`) and the jump detection used `horiz > 0.65 m`. On a square 1.2 × 1.2 m platform, a robot walking diagonally at 45° reaches `horiz = 0.65 m` at coordinates (0.46, 0.46) — still 14 cm inside the platform corner. The agent exploited this by slowly traversing to the corner over 450 steps, collecting near-maximum `edge_reward` per step, triggering the one- time jump bonus at the premature threshold, and then falling off the physical edge with only -50 steps remaining — insufficient time to complete the landing sequence. This geometry fix was applied before Configs 2 and 3 were trained.

Why Config 2 failed

Config 2's learning rate of 1×10^{-3} with batch size 128 caused systematic critic divergence. In SAC, the critic (Q-network) is trained using Bellman targets computed from the replay buffer. A high learning rate with small batch size produces noisy gradient estimates, causing the Q-values to overshoot and destabilise. Once the critic is inaccurate, the actor receives incorrect policy gradient signals and the entire learning process stalls.
The flat reward curve at –1.7 across 9,308 episodes and 500k timesteps confirms the policy never escaped this failure mode. The reduced discount factor ($\gamma = 0.95$ vs 0.99) further shortened the agent's planning horizon, making it harder to learn the multi-step walk-then-jump sequence.

Key lessons from hyperparameter tuning

1. **Entropy coefficient:** Auto-tuning (`ent_coef="auto"`) consistently outperformed fixed entropy, producing smoother convergence and a more capable final policy. Fixed `ent_coef=0.1` may be appropriate when the reward landscape is well-understood but is fragile otherwise.

2. **Batch size and learning rate must be co-tuned:** Large batches (512) stabilise the critic by reducing gradient variance, allowing higher or lower learning rates to be used safely. Small batches (128) require a correspondingly conservative learning rate to avoid divergence — the combination used in Config 2 (lr=1e-3, batch=128) was too aggressive.

3. **Reward shaping geometry matters:** Reward functions designed with circular assumptions applied to square environments introduce exploitable gradients. Platform geometry (square vs circular boundary)

must be reflected in both the jump detection threshold and the shaping signal to prevent the agent from finding unintended high-reward trajectories.

4. **Episode budget vs task complexity:** The three-phase task (locomote → jump → land) requires the agent to complete a long behavioural sequence. With a fixed 500-step budget, an agent that spends 200 steps reaching the edge has only 300 steps left for free-fall (~22 steps) and stable landing (30 steps required). Tighter stagnation termination (50 steps per band) successfully shortened unproductive episodes but the orientation control during flight remains an unsolved sub-problem within 500k training steps.